

Java Lab Report: Method Overriding & Dynamic Polymorphism

Course Outcome: CO1 (Object-Oriented Programming Fundamentals)

Topic: Method Overriding & Dynamic Polymorphism

1. Question / Problem Statement

Scenario: All the banks operating in India are controlled by RBI (e.g., minimum interest rate, minimum balance allowed, maximum withdrawal limit, etc.) which all banks must follow. For example, suppose RBI has set minimum interest rate applicable to a saving bank account to be 4% annually. However, banks are free to use 4% interest rate or to set any rates above it.

Task: Write a JAVA program to implement bank functionality in the above scenario and demonstrate the dynamic polymorphism concept. Note: Create a few classes namely Customer, Account, RBI (Base Class) and a few derived classes (SBI, ICICI, PNB etc). Assume and implement required member variables and functions in each class.

Test Cases:

Testcase 1:

Enter the Bank name to find the rate of Interest : RBI

RBI rate of interest is : 4%

Testcase 2:

Enter the Bank name to find the rate of Interest : SBI

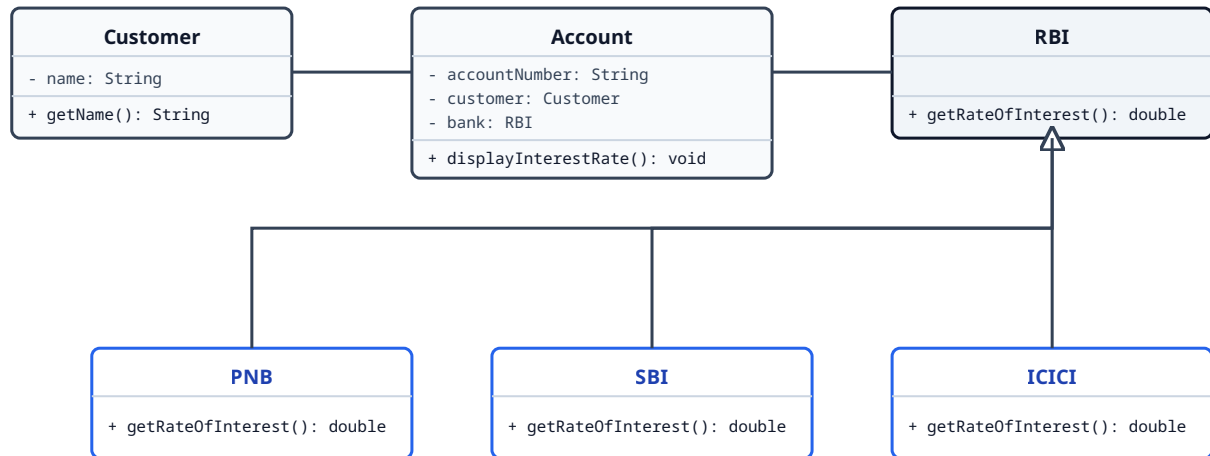
RBI rate of interest is : 7%

2. Step-by-Step Implementation Process

- 1. Base Class Creation (RBI):** Define the superclass RBI containing a baseline method `getRateOfInterest()` returning the default minimum rate of 4.0%.
- 2. Method Overriding in Derived Classes:** Create subclass entities (SBI, ICICI, PNB) extending RBI. Override the `getRateOfInterest()` method in each derived class to return bank-specific interest rates (e.g., SBI returns 7.0%, ICICI returns 6.5%, PNB returns 5.5%).
- 3. Domain Modeling (Customer & Account):** Implement the Customer class to encapsulate user entity information and the Account class to manage customer account attributes alongside a reference variable of type RBI.
- 4. Dynamic Binding Implementation:** In the Bank main class, create a base class reference variable (RBI bank). Dynamically assign subclass instances based on user input at runtime to demonstrate runtime dynamic polymorphism / dynamic method dispatch.

3. UML Diagrams

Class Diagram for Bank System Dynamic Polymorphism



4. JAVA Code Sample

```
package record;
import java.util.Scanner;

// Base Class
class RBI {
    // Default RBI interest rate
    public double getRateOfInterest() {
        return 4.0;
    }
}

// Derived Class 1
class SBI extends RBI {
    // Method Overriding
    @Override
    public double getRateOfInterest() {
        return 7.0;
    }
}

// Derived Class 2
class ICICI extends RBI {
    // Method Overriding
    @Override
    public double getRateOfInterest() {
        return 6.5;
    }
}

// Derived Class 3
class PNB extends RBI {
    // Method Overriding
    @Override
    public double getRateOfInterest() {
        return 5.5;
    }
}

// Customer details class
class Customer {
    private String name;

    public Customer(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

// Account details class using composition with RBI
class Account {
    private String accountNumber;
    private Customer customer;
    private RBI bank;

    public Account(String accountNumber, Customer customer, RBI bank) {
        this.accountNumber = accountNumber;
        this.customer = customer;
    }
}
```

```

        this.bank = bank;
    }

    public void displayInterestRate() {
        // Dynamic binding happens here based on the bank object passed
        System.out.println("RBI rate of interest is : " + (int)bank.getRateOfInterest() + "%");
    }
}

// Main class to demonstrate Dynamic Polymorphism
public class Bank {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the Bank name to find the rate of Interest : ");
        String bankName = scanner.next();

        // Base class reference variable (Demonstrates Dynamic Polymorphism)
        RBI bank;

        // Dynamic binding based on input
        if (bankName.equalsIgnoreCase("SBI")) {
            bank = new SBI();
        } else if (bankName.equalsIgnoreCase("ICICI")) {
            bank = new ICICI();
        } else if (bankName.equalsIgnoreCase("PNB")) {
            bank = new PNB();
        } else {
            bank = new RBI(); // Defaults to RBI standard
        }

        // Output matching the required test cases
        System.out.println("RBI rate of interest is : " + (int) bank.getRateOfInterest() + "%");

        scanner.close();
    }
}

```

5. Sample Program Output

Testcase 1 Output:

```

Enter the Bank name to find the rate of Interest : RBI
RBI rate of interest is : 4%

```

Testcase 2 Output:

```

Enter the Bank name to find the rate of Interest : SBI
RBI rate of interest is : 7%

```

Additional Testcase (ICICI Output):

```

Enter the Bank name to find the rate of Interest : ICICI
RBI rate of interest is : 6%

```